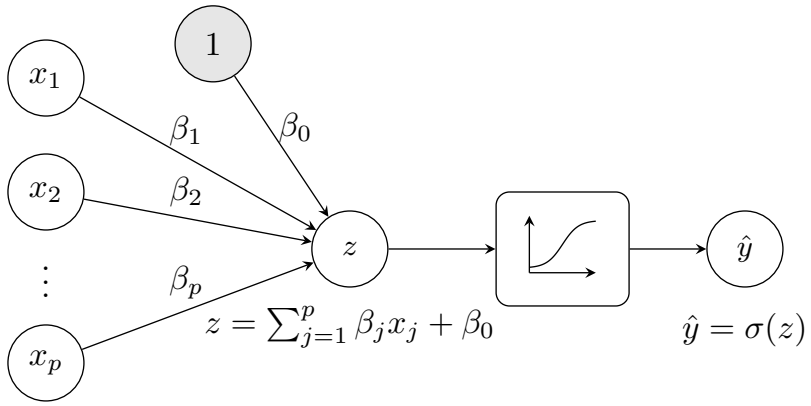
Ukesoppgaver - Uke 6

Logistisk regresjon - IN1160

I denne ukens ukesoppgaver skal vi regne oss gjennom noen eksempler for logistisk regresjon, tilsvarende slik vi gjorde med lineær regresjon i uke 5. Dette vil hjelpe med forståelsen av modellen og notasjonen, i tillegg til at det er relevant for eksamen. I oblig 2a vil dere implementere både logistisk og lineær regresjon med NumPy i Python. Det er anbefalt å gjøre ukesoppgavene for uke 5 før denne ukens oppgaver.

Mens lineær regresjon er en modell for regresjonsproblemer, der man prøver å predikere et hvilket som helst tall på tallinjen, er logistisk regresjon for binær klassifisering (tross "regresjon" i navnet, som dessverre er misveiledende). Det vil si at vi har to distinkte klasser som vi prøver å klassifisere inputene våre i. Vi noterer disse klassene med 0 og 1, men de kan representere hva som helst, for eksempel "katt" og "hund".

La oss igjen starte med å se på en visuell fremstilling av logistisk regresjon med diagramet i Figur 1.



Figur 1: Illustrasjon av en logistisk regresjonsmodell.

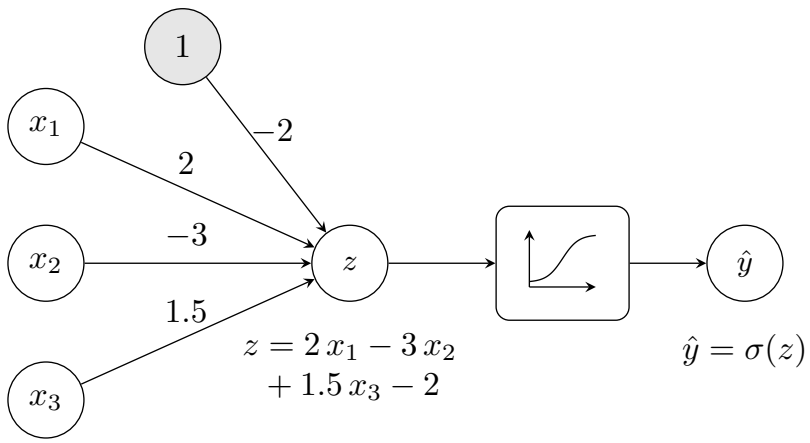
Bildet representerer altså ett datapunkt med p trekk og koeffisientene $(\beta_0, \beta_1, ..., \beta_p)$, der β_0 er konstantleddet.

Tilsvarende lineær regresjon er det én koeffisient per trekk i inputet og et konstantledd som blir ganget og plusset sammen. Istedenfor at denne summen blir prediksjonen $\hat{y}$, gir vi den navnet z, som vi kaller for den *vektede summen* (weighted sum). Dette blir så sent inn i sigmoid-funksjonen (representert med den greske bokstaven σ uttalt "sigma") som gir oss outputet $\hat{y}$. Dette er et tall som ligger mellom 0 og 1. For å få den endelige klassifikasjonen runder vi av til 0 eller 1 gitt en terskelverdi, ofte satt til 0.5.

Merk: Gitt naturen av funksjonene som blir brukt i logistisk regresjon er det vanskelig å lage oppgaver med runde tall. Det blir derfor litt flere desimaler å forholde seg til, så det lønner seg å bruke kalkulator (dere kan også starte et interaktivt miljø i Python ved å skrive `python` i terminalen). Det er greit å runde av svarende til to desimaler under utregningen.

Oppgave 6.1 - Vektet sum

La oss se på en spesifikk modell representert med illustrasjonen i Figur 2.



Figur 2: Eksempel på en spesifikk logistisk regresjonsmodell.

Vi har altså $p = 3$ antall trekk, og koeffisientene $(-2, 2, 3, -1.5)$.

Regn ut den vektede summen z for følgende datapunkter:

a) $\mathbf{x}_1 = (3, 2, 2)$

b) $\mathbf{x}_2 = (1.5, 0, -2)$

Oppgave 6.2 - Sigmoid

Vi kan nå bruke de vektede summene til å regne ut outputet av modellen. Dette gjør vi ved å sende de vektede summene gjennom sigmoid-funksjonen, som er definert ved:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Der e er den matematiske konstanten $e \approx 2.718$. Det er mulig å bruke `np.exp(-z)` for å regne ut e^{-z}.

Bruk nå den vektede summen z_1 og z_2 fra oppgave 6.1 a) og b) til å regne ut outputet $\hat{y}_1$ og $\hat{y}_2$

a) Hva blir $\hat{y}_1$? Dere kan runde av svaret til to desimaler.

b) Hva blir $\hat{y}_2$?

Oppgave 6.3 - Tapsfunksjon

Vi kan nå bruke outputet vårt til å regne ut tapsfunksjonen (loss-funksjonen) vår. For logistisk regresjon bruker *binær kryssentropi*, ofte referert til som *binary cross entropy* (BCE), som er definert ved:

$$\mathrm{BCE}(\mathbf{y}, \hat{\mathbf{y}}) = -\frac{1}{n} \sum_{i=1}^{n} \left(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right)$$

La oss raskt se på hvordan denne funksjonen fungerer. For datapunkt nummer i er den sanne verdien y_i enten lik 0 eller 1. Dersom den er 0, blir leddet $y_i \log(\hat{y}_i)$ lik 0 og faller bort, og dersom y_i er lik 1 blir leddet $(1 - y_i) \log(1 - \hat{y}_i)$ lik 0 og faller bort. Vi får altså ett ikke-null ledd for hvert datapunkt. Outputet $\hat{y}_i$ ligger mellom 0 og 1. Dersom inputet til logaritmen går mot 0, vil logaritmen gå mot minus uendelig. Dersom inputet er 1, vil logaritmen være 0.

Dette betyr at dersom vi har $y_i = 1$ vil tapet være lavt dersom $\hat{y}_i$ er nærme 1, og veldig høyt (siden vi har et minus tegn i starten av funksjonen) om $\hat{y}_i$ er nærme 0. For $y_i = 0$ får vi den motsatte situasjonen: Tapet blir høyt når $\hat{y}_i$ er nærme 1 og lavt når den er nærme 0.

a) Gitt de sanne verdiene $\mathbf{y} = (1, 0)$, hva blir tapsfunksjonen for prediksjonene i oppgave 6.2?

b) Hva ville tapsfunksjonen vært om de sanne verdiene var $\mathbf{y} = (0, 1)$ isteden? Gir denne verdien mening i forhold til verdien fra a)?

Oppgave 6.4 - Nøyaktighet (accuracy)

Tapsfunksjonen er nyttig for å trene modeller, men den gir ikke et enkelt tolkbart tall på hvor bra modellen vår gjorde det. Dette gjør derimot *nøyaktighet*, ofte kalt

accuracy. Nøyaktighet er definert ved antall riktig klassifiserte datapunkter delt på antallet av alle datapunkter. Vi kan notere dette som:

$$\text{accuracy}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{n} \sum_{i=1}^{n} I(y_i = \hat{y}_i)$$

der I er indikatorfunksjonen, som returnerer 1 dersom uttrykket inne i funksjonen er sant, og 0 ellers.

Får å få en prediksjon må vi runde av outputet til enten 0 eller 1. Den enkleste og vanligste måten å gjøre dette på er å runde opp til 1 dersom outputet er høyere enn en terskelverdi, og ned ellers. Som oftest er terskelverdien satt til 0.5, men vi kan endre den for å endre hvordan modellen oppfører seg. Vi bruker prediksjonene fra oppgave 6.2.

a) Regn ut nøyaktigheten dersom $\mathbf{y} = (1, 1)$ og terskelverdien er 0.5

b) Regn ut nøyaktigheten dersom $\mathbf{y} = (0, 0)$ og terskelverdien er 0.5

c) Regn ut nøyaktigheten dersom $\mathbf{y} = (0, 0)$ og terskelverdien er 0.8

d) Kan du komme på noen situasjoner der det ville vært gunstig å ha en terskelverdi forskjellig fra 0.5?

Oppgave 6.5 - Optimalisering av koeffisientene

Tilsvarende som med lineær regresjon vil vi finne koeffisienter som *minimerer tapsfunksjonen vår*. Dette gjør vi ved å *derivere tapsfunksjonen med hensyn på koeffisientene våre* for å finne ut hvilken retning vi burde oppdatere dem i, for så å bruke *gradient nedstigning* (gradient descent) til å gjøre en liten oppdatering av koeffisientene i denne retningen. Denne prosessen er nøyere beskrevet i ukesoppgaver for uke 5 og i oblig 2a.

For at oppdateringen av koeffisientene skal bli riktig må vi bruke derivasjonsregler til å regne ut de deriverte (igjen er ikke selve utregning del av pensum). Det viser seg at sigmoid-funksjonen og BCE "kansellerer" hverandre, og vi får samme oppdateringsregel som ved lineær regresjon (husk allikevel at $\hat{y}$ er regnet ut på forskjellige måter). Altså har vi:

$$\beta_0^{(k)} \leftarrow \beta_0^{(k-1)} - \eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

for konstantleddet, og for de andre koeffisientene β_j der $j \neq 0$ får vi:

$$\beta_j^{(k)} \leftarrow \beta_j^{(k-1)} - \eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_{i,j}$$

Disse formlene er viktige, så la oss øve litt mer på å bruke dem. Utregningen blir noe slitsom for hånd, siden tallene fra sigmoid ikke blir runde. La oss derfor bruke noen andre tall, og anta at vi har inputene $\mathbf{x}_1 = (1, 2)$ og $\mathbf{x}_2 = (5, -3)$ med sanne verdier $\mathbf{y} = (0, 1)$ med prediksjoner fra en annen modell $\hat{y}_1 = 0.2$ og $\hat{y}_2 = 0.3$. Anta også at $\eta = 0.1$.

Vi ser på vektdifferansen $\Delta\beta_j$, som er leddet i oppdateringsregelen med læringsraten η i seg, altså $\eta \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_{i,j}$, med $x_{i,0} = 1$ for konstantleddet β_0.

a) Gitt antagelsene over, regn ut $\Delta\beta_0$.

b) Regn ut $\Delta\beta_1$.

c) Regn ut $\Delta\beta_2$.